

Իրականացնել հետևյալ **Descriptor**-ները, որոնք թույլ կտան կառավարել կլասի ատրիբուտները, ապահովել տվյալների ամբողջականությունը և ավտոմատացնել վալիդացիայի գործընթացը:

1. Թվային և տեքստային սահմանափակումներ

- **PositiveNumber:** Ստեղծել descriptor, որը թույլ կտա սահմանել միայն 0-ից բարձր թվային արժեքներ: Հակառակ դեպքում բարձրացնել `ValueError`:
- **MaxLength:** Ստեղծել descriptor, որը կընդունի `max_len` պարամետրը և թույլ չի տա տողին գերազանցել այդ երկարությունը:

2. Տիպերի ստուգում և լռելային արժեքներ

- **IntegerField:** Descriptor, որը կստուգի, որ տրված արժեքը բացառապես լինի `int` տիպի (`TypeError`):
- **Default Field:** Ստեղծել Field descriptor, որը հնարավորություն կտա սահմանել `default` արժեք: Եթե օբյեկտի attribute-ը դեռ սահմանված չէ, այն պետք է վերադարձնի այդ լռելային արժեքը:

3. Հասանելիության վերահսկում

- **ReadOnlyField:** Իրականացնել descriptor, որը թույլ է տալիս արժեքը սահմանել միայն մեկ անգամ (օրինակ՝ ID-ների համար): Կրկնակի փոփոխության փորձի ժամանակ բարձրացնել `AttributeError`:

4. Փոփոխության հետևում

- **TrackedField:** Ստեղծել descriptor, որն ամեն անգամ արժեքը փոփոխելիս այդ attribute-ի անունը կավելացնի օբյեկտի ներքին `_changed_fields` բազմության մեջ:

5. Վալիդացիայի համակարգի նախագծում

- Ստեղծել բազային դաս՝ **BaseField**, որն ունի `validate` մեթոդ:
- Ժառանգել `BaseField`-ից հետևյալ դասերը.
 - **StringField** (ստուգում է տիպը),
 - **IntegerField** (ստուգում է տիպը),
 - **EmailField** (ստուգում է տիպը և արդյոք պարունակում է @ և . նշանները):

Սպասվող արդյունքի օրինակ (Test Cases)

```
class Profile:
    uid = ReadOnlyField()
```

```
age = IntegerField()
email = EmailField()
status = Field(default="Active")
balance = PositiveNumber()

u = Profile()
u.uid = 1001          # OK
# u.uid = 1002        # Պետք է առաջացնի AttributeError
u.age = 25            # OK
# u.age = "25"        # Պետք է առաջացնի TypeError
u.email = "test@mail.com" # OK
print(u.status)      # Պետք է տպի "Active"
```